Project Title

Mini Search Engine Using Data Structures and Information Retrieval

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
B.Siddhartha-2500090225
C.V Jaswanthreddy-2520080025
Team Number: 10

Section: 12

Under the guidance of

Ms.K.Chandusha

Assistant Professor, CS&IT



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY/ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DECLARATION

The Mini Project Report entitled "**Mini Search Engine Using Data Structures and Information Retrieval"** is a record of Bonafide work of **B.Siddhartha– 2500090225, C.V Jaswanthreddy – 2520080025.** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering/Computer Science and Information Technology/ Artifical Intelligence & data Science to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

B.Siddhartha – 2500090225

C.V Jaswanthreddy – 2520080025

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled "**Mini Search Engine Using Data Structures and Information Retrieval"** is a bonafide work done and submitted by **B.Siddhartha – 2500090225, C.V Jaswanthreddy – 2520080025, .** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering/ Computer Science and Information Technology/Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, <<Guide Name>>, <<Designation>>, <Department>> which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Rao (CSE)/Dr. Khaja Shareef (CSIT)/ Dr. Sreenath Itikela (AI&DS)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology/Department of Artificial Intelligence and Data Science* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

B.Siddhartha– **2500090225.**

C.V Jaswanthreddy – 25200800225.

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	
2.	Introduction	
3.	Literature survey	
4.	Problem Statement and Objectives	
5.	Hardware and Software requirements	
6.	Implementation	
7.	Results & Discussion	
8.	Conclusion & Future Scope	
9.	References	

1.ABSTRACT

Efficient text retrieval relies heavily on the optimal choice of underlying datastructures and algorithm design. This project details the architectural design, algorithmic implementation, and complexity analysis of a localized Mini Search Engine Using Data Structures and Information Retrieval engineered to process, index, and query unstructured text documents.The system demonstrates the direct practical application of foundational Data Structures and Algorithms (DSA)—specifically Hash Tables, Dynamic Arrays, Linked Lists, Trie structures, and Sorting algorithms—in solving real-world information retrieval problems.At the core of the engine is an **Inverted Index** implemented using a Hash Table mapping unique terms (keys) to sorted Posting Lists (values), enabling **O(1)** average-time keyword lookups. Document ingestion involves tokenization, normalization, and stop-word filtering.Query processing employs multi-way Merge operations and Intersection algorithms on posting lists to execute Boolean operations (AND, OR) in **O(n + m)** time. To present ranked search results, documents are evaluated using **Term Frequency-Inverse Document Frequency (TF-IDF)** metrics combined with Quick Sort / Heap Sort algorithms, achieving optimal **O(k log N)** extraction for top-k relevant results.

Team Name:

Team Logo (if any):

Team Members:

1.B.Siddhartha-2500090225

2.C.V Jaswanthreddy-2520080025

2. INTRODUCTION

In today's digital world, a huge amount of information is stored in the form of text documents, making fast and efficient information retrieval an important requirement. Search engines help users find relevant information from large collections of documents by processing queries and ranking the results according to their relevance. This project, **"Mini Search Engine Using Data Structures and Information Retrieval,"** focuses on developing a localized search engine that demonstrates how fundamental **Data Structures and Algorithms (DSA)** can be applied to solve real-world search and information retrieval problems.The main objective of this project is to design a search system that can **process, index, search, and rank text documents efficiently**. The system uses an **Inverted Index** to map keywords to the documents in which they occur, allowing faster keyword searches. Various data structures such as **Hash Tables, Dynamic Arrays, Linked Lists, and Trie structures** are used to organize and manage the data efficiently.The project also applies important information retrieval techniques such as **tokenization, text normalization, stop-word removal, Boolean searching, and TF-IDF-based ranking**. Algorithms such as **Merge Sort, Quick Sort, Heap Sort, and posting-list intersection** are incorporated to improve search performance and rank the most relevant documents.Overall, this project provides a practical understanding of how **DSA and Information Retrieval techniques work together to build an efficient search engine**, while also analyzing the time and space complexity of the implemented algorithms.

3. LITERATURE SURVEY

Dear students, in Literature Survey section of the Hackathon Project Documentation you need to specify the previous similar kind of projects already done with an explanation in a 4-to-5-line paragraph and in References section of documentation you need to specify the URLs of the similar kind of websites which you referred and included in the Literature. Minimum of 5 to 10 similar kind of project you need to refer and do the literature and put the URL links in References.

4. Problem Statement and Objectives

Problem Statement: Develop a Mini Search Engine using Data Structures and Information Retrieval techniques to efficiently index, search, and retrieve relevant information from a collection of stored files.

Problem Description: Searching for specific words or information in multiple files manually is time-consuming and inefficient. The project aims to provide fast and accurate searching by indexing files and applying string-matching, ranking, and similarity algorithms.

Objectives:

The project is initiated to build a simple and efficient search system that demonstrates how **Data Structures and Algorithms (DSA)** can be used for real-world information retrieval.

Main Objectives

1. **Create an efficient index** of words and documents for faster searching.
2. **Search for keywords** across multiple stored files efficiently.
3. **Apply string-matching algorithms** such as KMP, Rabin-Karp, Aho-Corasick, and Z Algorithm.
4. **Improve search accuracy** using similarity and fuzzy-matching techniques.
5. **Rank search results** based on relevance and similarity.
6. **Reduce search time** compared with manually scanning each file.

7. **Provide a user-friendly interface** for uploading, storing, and searching files.
8. **Demonstrate practical applications of DSA and Information Retrieval** in a search engine.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- **Processor:** Intel Core i3 or equivalent processor and above.
- **RAM:** Minimum 4 GB RAM; 8 GB recommended for better performance.
- **Storage:** Minimum 10 GB of free disk space for storing project files, documents, and database data.

Software Requirements:

- **Programming Language & IDE:** Java and Visual Studio Code (VS Code).
- **Database:** MongoDB / MongoDB Compass for storing file and indexing information.

- **Web Technologies:** HTML, CSS, and JavaScript for the frontend interface.
- **Backend:** Java-based backend for implementing search and data-structure algorithms.
- **Version Control:** Git and GitHub for project management and source-code storage.
- **Required Algorithms:** KMP, Rabin-Karp, Aho-Corasick, Z Algorithm, Suffix Array, Suffix Automaton, Levenshtein Distance, Damerau-Levenshtein, and Cosine Similarity.

6. IMPLEMENTATION

Module 1: File Indexing Module

This module reads the uploaded files and creates an index of the words contained in them. The index helps the search engine locate relevant files quickly.

```
Map<String, List<String>> invertedIndex = new HashMap<>();

public void indexFile(String fileName, String content) {
    String[] words = content.toLowerCase().split("\\W+");

    for (String word : words) {
        if (!word.isEmpty()) {
            invertedIndex
                .computeIfAbsent(word, k -> new ArrayList<>())
                .add(fileName);
        }
    }
}
```

Module 2: Keyword Search Module – KMP Algorithm

The KMP (Knuth-Morris-Pratt) algorithm is used to efficiently search for a keyword or pattern inside the file content.

```
public boolean kmpSearch(String text, String pattern) {
    text = text.toLowerCase();
    pattern = pattern.toLowerCase();

    int[] lps = new int[pattern.length()];
    computeLPS(pattern, lps);

    int i = 0, j = 0;

    while (i < text.length()) {
        if (text.charAt(i) == pattern.charAt(j)) {
            i++;
            j++;

            if (j == pattern.length())
                return true;
        } else if (j > 0) {
            j = lps[j - 1];
        } else {
            i++;
        }
    }
    return false;
}

private void computeLPS(String pattern, int[] lps) {
    int len = 0;

    for (int i = 1; i < pattern.length();) {
        if (pattern.charAt(i) == pattern.charAt(len)) {
            lps[i++] = ++len;
        } else if (len > 0) {
            len = lps[len - 1];
        } else {
            lps[i++] = 0;
        }
    }
}
```

Module 3: Similarity and Fuzzy Search Module

This module compares the search query with document text and helps find results even when there are spelling differences or similar words. Algorithms such as **Levenshtein Distance, Damerau-Levenshtein, and Cosine Similarity** are used.

```
public int levenshteinDistance(String a, String b) {
    int[][] dp = new int[a.length() + 1][b.length() + 1];

    for (int i = 0; i <= a.length(); i++)
        dp[i][0] = i;

    for (int j = 0; j <= b.length(); j++)
        dp[0][j] = j;

    for (int i = 1; i <= a.length(); i++) {
        for (int j = 1; j <= b.length(); j++) {

            int cost = (a.charAt(i - 1) == b.charAt(j - 1)) ? 0 : 1;

            dp[i][j] = Math.min(
                Math.min(dp[i - 1][j] + 1,
                         dp[i][j - 1] + 1),
                dp[i - 1][j - 1] + cost
            );
        }
    }

    return dp[a.length()][b.length()];
}
```

7. RESULTS & DISCUSSION

The implemented Mini Search Engine successfully indexes files and retrieves documents based on the user's search query. The system demonstrates the practical use of **Data Structures, String Matching, Fuzzy Searching, and Information Retrieval techniques**.

Screenshot 1: Home/Search Page

Explanation: The home page provides the interface for uploading files and entering a search query. Users can enter keywords to search across the stored documents.

Screenshot 2: File Upload / Indexing

Explanation: The selected files are uploaded and processed by the system. The indexing module extracts words from the files and stores them in the search index.

Screenshot 3: Search Results

Explanation: The search engine displays the documents containing the searched keyword. Matching documents are retrieved efficiently using the implemented search algorithms.

8. CONCLUSION & FUTURE SCOPE

Conclusion:

The Mini Search Engine Using Data Structures and Information Retrieval successfully provides an efficient way to index, search, and retrieve information from multiple files. The project demonstrates the practical application of DSA algorithms such as KMP, Rabin-Karp, Aho-Corasick, Z Algorithm, Suffix Array, Levenshtein Distance, and Cosine Similarity. The main objectives of fast keyword searching, relevant result retrieval, and improved search accuracy were achieved. This project also helped demonstrate how data structures and string-processing algorithms can be applied to develop a real-world search engine.

Future Scope:

The future scope of the Mini Search Engine Using Data Structures and Information Retrieval includes improving the system to handle large collections of documents and different file formats such as PDF and DOCX. The search engine can be enhanced with advanced ranking, spelling correction, fuzzy matching, and semantic search to provide more accurate and relevant results. In the future, Artificial Intelligence and Natural Language Processing (NLP) can be integrated to understand the meaning and context of user queries. The system can also be extended with cloud storage, real-time indexing, and scalable database technologies to make it suitable for large datasets and real-world applications.

9. REFERENCES

1. **Apache Lucene**
2. **Apache Solr**
3. **Elasticsearch**
4. **OpenSearch**
5. **Whoosh**
6. **Meilisearch**
7. **Typesense**